



Arquitectura del sistema — SIGEZOO

Rol 2 — Arquitecto de Software

1. Eleccion y justificacion del stack tecnologico

Componente	Eleccion	Justificacion
Sistema operativo (desarrollo)	Windows + Laragon	Entorno que ya usa el equipo; Laragon integra Apache, MySQL y PHP sin configuracion manual, lo que agiliza el desarrollo del prototipo.
Sistema operativo (produccion recomendado)	Linux (Ubuntu Server)	Estandar de facto para hosting de aplicaciones PHP/MySQL: menor costo de licenciamiento, mejor soporte de la comunidad Laravel y mayor estabilidad para procesos de larga duracion.
Servidor web	Apache HTTP Server	Incluido en Laragon, configuracion por .htaccess ya soportada por Laravel out-of-the-box, y es el servidor con el que el equipo tiene mas experiencia.
Servidor de aplicaciones	PHP 8.3 (mod_php en desarrollo)	Motor de ejecucion de Laravel. En produccion se recomienda migrar a PHP-FPM detras de Nginx o Apache, ya que maneja mejor la concurrencia de multiples usuarios.
Base de datos	MySQL 8.4	Motor relacional gratuito y ampliamente soportado. El dominio (habitat -> animal -> limpieza/alimentacion/clinico) tiene relaciones claras y consultas cruzadas (reportes), lo que encaja con un modelo relacional normalizado en 3FN. Soporte de primera clase en Laravel (Eloquent ORM).
Framework	Laravel 11 (PHP)	Provee routing, ORM, autenticacion, validacion y proteccion CSRF integradas, reduciendo el codigo de infraestructura necesario para cumplir el requisito de "aspectos basicos de seguridad" del enunciado.

2. Estilo arquitectonico

Monolito en capas (MVC), con separacion de modulos por dominio.

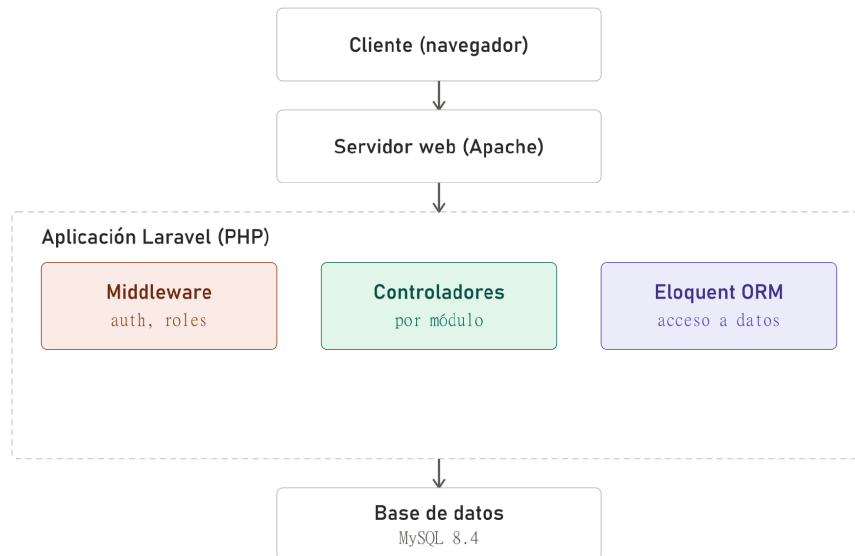
Se descarto una arquitectura de microservicios porque:

- El equipo es de 5 personas con un plazo corto (entrega 26/09/2026); microservicios anadirian complejidad de despliegue (orquestracion, comunicacion entre servicios) que no aporta valor a un prototipo academico.
- El sistema es de uso interno de un solo zoologico, sin necesidad de escalar modulos de forma independiente.

Dentro del monolito, el codigo se organiza en capas: rutas -> middleware (autenticacion y roles) -> controladores por modulo -> modelos / Eloquent ORM -> base de datos MySQL.

Cada modulo funcional (Limpieza, Alimentacion, Control Clinico, Entradas) vive en su propia carpeta de controladores y vistas, lo que permite que cada integrante del equipo trabaje su modulo sin pisar el codigo de los demas, aunque todos corran sobre la misma aplicacion Laravel.

3. Diagrama general de arquitectura



Componentes:

- Cliente (navegador): consume la interfaz Blade renderizada por el servidor. Sin SPA/API separada, server-side rendering clasico de Laravel.
- Servidor web (Apache): recibe las peticiones HTTP y las enruta al front controller de Laravel (public/index.php).
- Middleware de autenticacion y roles: primer filtro de seguridad antes de llegar a cualquier controlador de modulo.
- Controladores por modulo: logica de negocio de cada area (limpieza, alimentacion, control clinico, entradas).
- Eloquent ORM: capa de acceso a datos, traduce modelos PHP a consultas SQL.
- Base de datos MySQL: persistencia de todas las entidades del sistema, con el habitat/animal como eje central del modelo.

4. Por qué se eligió esta arquitectura

- Simplicidad sobre el requisito real: el enunciado pide un sistema simple y funcional con conexion a BD, seguridad basica y reportes, un monolito en capas cubre esto sin sobre-ingenieria.
- Curva de aprendizaje del equipo: Laravel resuelve routing, autenticacion, ORM y proteccion CSRF de fabrica, dejando que el equipo se enfoque en la logica de negocio de cada modulo.



- Desarrollo distribuido sin bloqueos: al separar los modulos por carpetas y dejar el entorno base (conexion a BD, autenticacion, roles) montado desde el inicio, cada integrante pudo empezar su CRUD sin esperar a que otro terminara el suyo.
- Costo y despliegue: PHP + MySQL + Apache es una de las combinaciones de hosting mas economicas y ampliamente disponibles, adecuada para un prototipo academico.

5. Consideraciones de seguridad básica

- **Autenticacion:** Laravel Breeze (login, registro, recuperacion de contrasena) usando sesiones de servidor y contrasenas con hash bcrypt.
- **Control de roles:** tabla roles (administrador, operativo, visitante) relacionada a users via id_rol. Middleware role:administrador,operativo protege las rutas de los modulos internos.
- **Prevencion de escalacion de privilegios:** el formulario de registro publico siempre asigna el rol visitante en el servidor, el rol nunca se toma de un campo del formulario.
- CSRF: todos los formularios usan el token CSRF integrado de Laravel.
- **Validacion de entradas:** cada Request de Laravel valida tipo, longitud y formato antes de tocar la base de datos.
- **Secretos fuera del control de versiones:** credenciales de BD y APP_KEY viven unicamente en .env, excluido en .gitignore.
- **Autorizacion de acceso HTTP 403:** cualquier intento de acceder a una ruta sin el rol adecuado responde con 403 Forbidden.